Spring MVC Security - Custom Login Form – Coding

In this lesson, we will embark on the process of developing a custom login form in a Spring application. This will be a two-step process.

Step 1: Modifying the Spring Security Configuration

First, we'll modify the Spring Security Configuration to reference our custom login form. Let's dive into the source code under `src/main/java` and open up `DemoSecurityConfig`. Soon, you'll know how we configure the security for web paths, including login, logout, etc.

Our focus here will be the login portion, where we create a security filter bean, passing in HTTP security. Inside the method, we have `http.authorizeHttpRequest` followed by `.anyRequest().authenticated()`. This means any request must be authenticated, or in other words, a user must be logged in.



For the form login, we'll specify the actual login page, the login processing URL, and we'll use `permitAll`, which means everyone can access this without being logged in. Essentially, this is how you configure Spring Security to reference your custom login form.

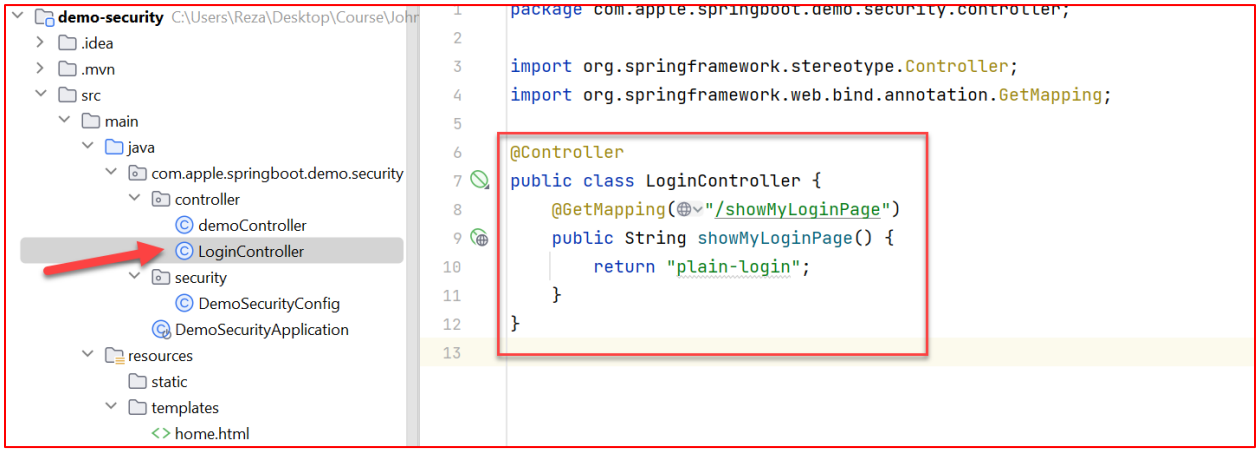
Step 2: Developing the Controller

Next, we need to create a controller for this request mapping, and then handle the `GetMapping` for `/showMyLoginPage`. Note that there's no controller request mapping required for the processing URL; Spring handles this for you behind the scenes.

Now that we have our method and configuration all set up, let's create a new class called `LoginController`. Here, we'll be dealing with regular Spring MVC coding.

We'll annotate the class with `@Controller` and then write the mapping as follows:

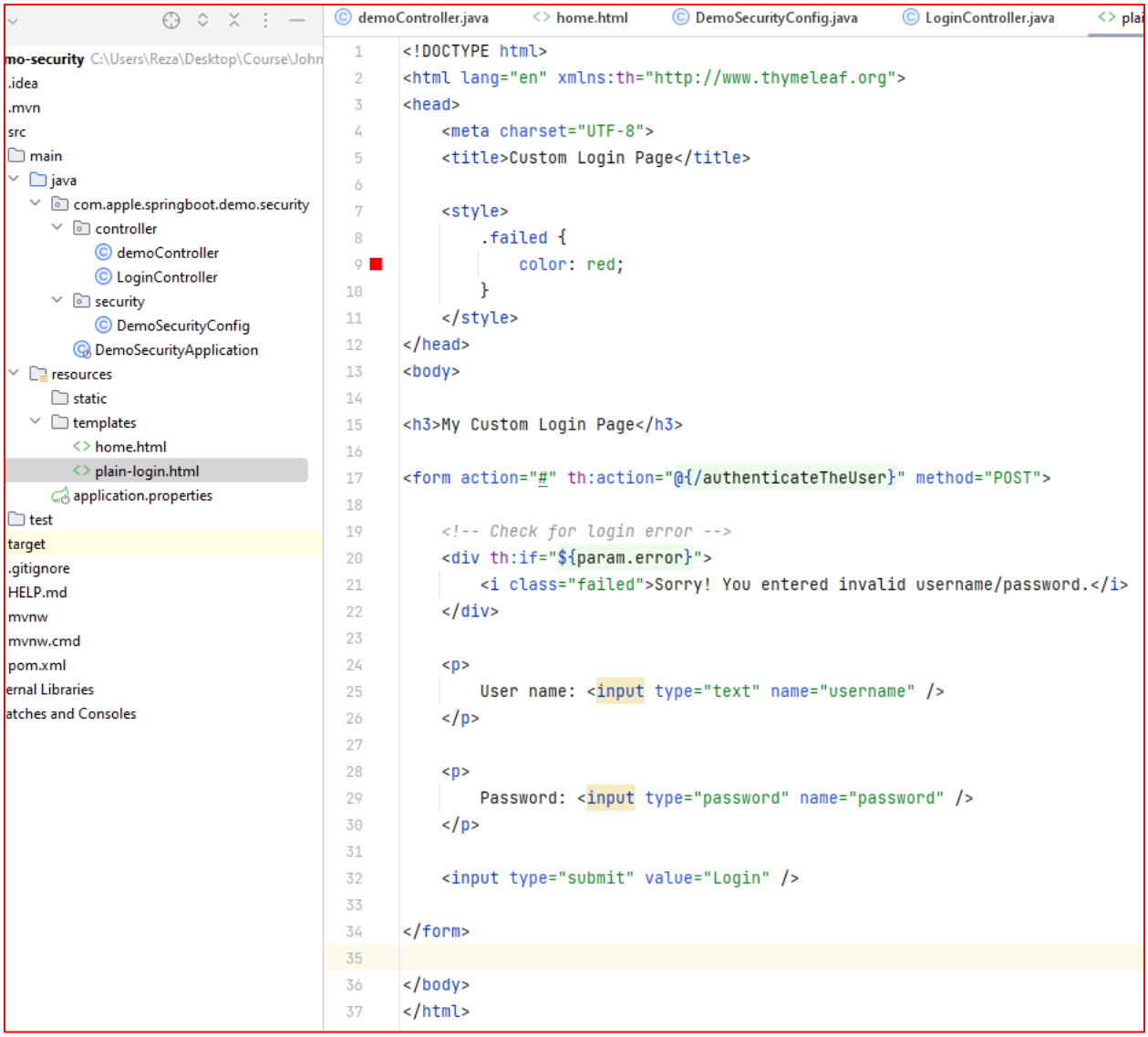
```
@GetMapping("/showMyLoginPage")
public String showMyLoginPage() {
    return "plain-login";
}
```



The information here must match up with our Spring security configuration file.

Next, we'll create the view named `plain-login` and design the HTML for our form. We'll add a title and set up the action as `/authenticateTheUser` with `method="POST"`. Let's also resolve any issues, like adding the Thymeleaf XML Namespace.

For the username and password fields, we'll use `input` types, and we must make sure that the names correspond to what Spring Security will look for. Lastly, we'll add a submit button with the value "Login."

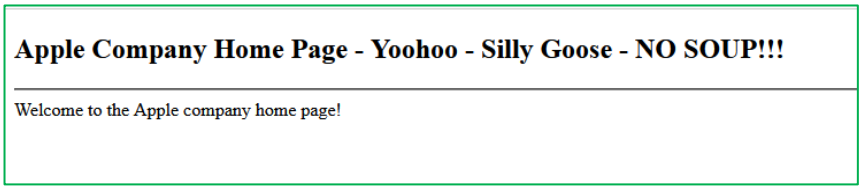


With our custom login form ready, let's remember that Spring Security filters will read the form data and authenticate the user using these default form field names. Ensure that you have the exact names.

Testing the Form

We're now at a point where we can run and test our form. After entering a user ID and password, we can see our custom login page configured with the Spring Security system.





However, there's an issue. If we enter incorrect credentials, there's no error message. This problem arises because we are creating our own custom login page and must add logic to handle login failure.